

Flux: Optimal Scheduling of Optical Circuit Switches for LLM Training

Arno Troch
IDLab, University of Antwerp - imec
Antwerp, Belgium
arno.troch@uantwerpen.be

Seyyidahmed Lahmer
imec
Leuven, Belgium
seyyidahmed.lahmer@imec.be

Abubakr Nada
imec
Leuven, Belgium
abubakr.nada@imec.be

Jeroen Famaey
IDLab, University of Antwerp - imec
Antwerp, Belgium
jeroen.famaey@uantwerpen.be

Michael Peeters
imec
Leuven, Belgium
michael.peeters@imec.be

Abstract

Optical Circuit Switching (OCS) offers high bandwidth density and energy efficiency for LLM training, but incurs a non-negligible reconfiguration delay. Prior work typically schedules optical circuit switches independently of compute, using aggregate traffic demand to determine which circuits to provision and when. We argue that this separation creates a fundamental inefficiency: reconfigurations that ignore the compute timeline can stall communication, resulting in low circuit utilization and large buffer requirements.

In this paper, we present FLUX, a scheduler that optimally schedules optical circuit switches based on the structure of the entire workload. FLUX remains effective across a wide range of switching speeds by reusing circuits and amortizing reconfiguration delay behind compute and communication. We show that FLUX reduces training iteration time by up to 10× and peak NIC buffer requirements by more than three orders of magnitude compared to traditional periodic schedulers.

1 Introduction

Large Language Model (LLM) training runs at massive scale. Modern training systems rely on hundreds of thousands of accelerators with frequent bandwidth-intensive collective communication [6, 22]. As models grow, these communication operations have become a primary bottleneck. Traditional electrical packet-switched networks are approaching fundamental limits in bandwidth density and power efficiency [8]. Optical Circuit Switching (OCS) offers deterministic, high-bandwidth connections without queuing delays or congestion [1, 16, 21]. Unlike packet switching, where individual packets are multiplexed across electrical switches, an optical circuit switch establishes direct optical paths between endpoints for the duration of a data transfer, without per-packet buffering or arbitration. However, OCS introduces a non-negligible reconfiguration overhead when changing

the network topology, so reconfigurations need to be intelligently scheduled.

To manage this overhead, prior work typically treats circuit scheduling as a purely networking problem. Approaches ranging from one-shot static topologies to periodic reconfiguration derive their schedules from an aggregate traffic matrix [4, 13, 15, 20, 24, 30, 33]. We argue that this assumption is a poor fit for modern LLM training workloads. A traffic matrix sums communication volume over a time window. Therefore, it discards the strict execution order of the training loop. LLM training, by contrast, is highly structured and deterministic, since communication operations occur at precise moments and depend on the completion of preceding compute and communication steps [14].

Furthermore, separating communication scheduling from compute creates an inefficiency. Schedulers that rely on traffic matrices cannot capture compute-communication overlap, which means that the network may be performing large transfers while accelerators sit idle waiting for data. Producing data without first confirming that the required circuit is available also increases end-host buffer requirements. This points to a gap: minimizing network completion time in isolation does not necessarily minimize the end-to-end iteration time of the training workload.

Unlike existing *demand-aware* approaches, which typically generate a schedule from the aggregate traffic volume a specific workload produces, we argue that circuit scheduling must directly account for the temporal structure of the entire workload. By taking into account the temporal dependencies of the workload, we can more effectively reuse circuits and hide reconfiguration latency behind compute phases.

To demonstrate this, we introduce FLUX, a framework that formalizes workload-aware circuit scheduling as a Mixed Integer Linear Program (MILP) that consolidates compute and communication into a single circuit schedule. We show that FLUX reduces training iteration time and buffer requirements by producing schedules that naturally lead to circuit reuse,

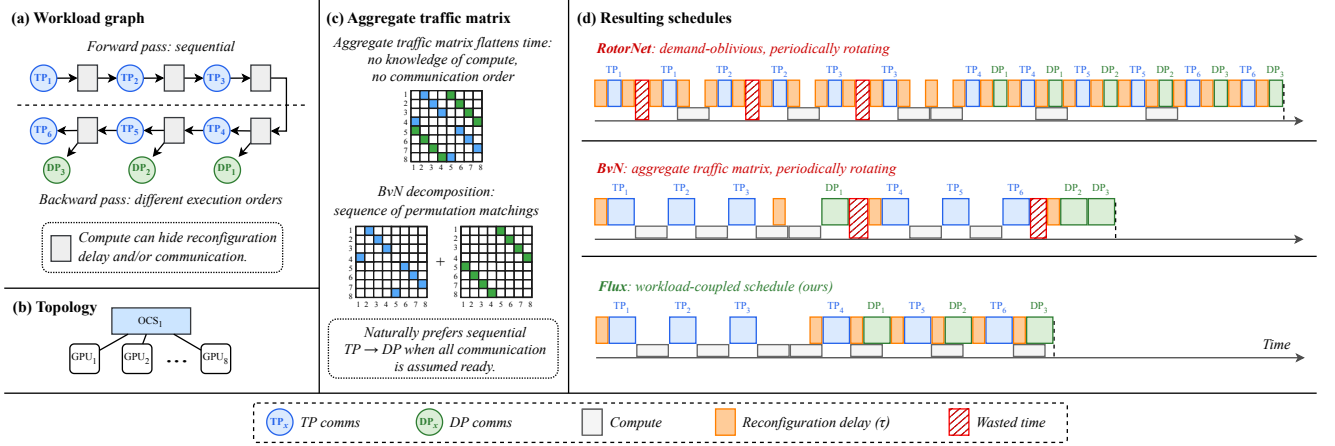


Figure 1: Depiction of how FLUX differs from existing circuit scheduling approaches. (a) an example training step (TP + DP) as a graph of compute and communication steps with dependencies (b) the network topology with 1 OCS switch and 8 GPUs; (c) an aggregate traffic matrix of the workload that records total communication volume but discards the graph ordering; (d) three circuit schedules for this workload. RotorNet [19] and BvN-based schedules [7, 29] reconfigure on a fixed period regardless of the workload’s state, stalling compute and communication when the active circuit is wrong. FLUX derives its schedule from the graph, reusing circuits and amortizing reconfiguration behind compute instead.

compute-communication overlap, and amortization of the reconfiguration delay. We make the following contributions:

- (1) We show why aggregate traffic matrix scheduling is a poor fit for the deterministic, dependency-structured nature of LLM training.
- (2) We formalize the circuit scheduling problem as a MILP and introduce FLUX, a workload-aware circuit scheduler that takes into account reconfiguration delay, compute-communication dependencies, and switch assignment.
- (3) We evaluate FLUX against traditional periodic schedulers and find that it consistently reduces iteration time across a range of reconfiguration delays, while significantly reducing the Network Interface Card (NIC) buffer requirements.

The remainder of the paper is organized as follows: Section 2 motivates workload-aware scheduling, Section 3 presents the MILP formulation, Section 4 reports the evaluation, Section 5 contrasts with related work, and Section 6 concludes and discusses avenues for future work.

2 Beyond Aggregate Traffic Matrices

OCS introduces a non-negligible reconfiguration delay, which makes deciding which topologies to transition to and when non-trivial. A natural approach to managing this overhead has been to offload scheduling to controllers that monitor demand, or to rely on periodic reconfigurations. In most cases, these systems build their topologies around an aggregate traffic matrix: a matrix where entry (i, j) gives the

total traffic volume sent from source i to destination j over a time window). This creates a fundamental limitation: A traffic matrix flattens time. It views the workload as a bulk data transfer problem rather than a sequence of pipelined transfers. This means that while the total communication volume is satisfied, the temporal structure of the LLM training iteration is completely lost.

Prior work often attempts to alleviate this gap through intelligent matrix decomposition. For example, a common strategy is to isolate large flows for dedicated optical circuits, while allowing smaller latency-sensitive flows to bypass the reconfiguration penalty via multi-hop routing or even a separate packet-switched network [15]. Alternative approaches heavily rely on Birkhoff-von Neumann (BvN) decomposition [7, 29] to partition the demand matrix into a convex combination of permutation matrices [4, 13, 15, 20, 24, 30, 33]. However, these techniques still operate on a flattened, aggregate view of demand: partitioning or decomposing the traffic matrix changes how demand is expressed. None of them recover the execution order of the underlying workload, which is especially important for LLM training.

We illustrate this disconnect in Figure 1. Consider a training step distributed across eight Graphics Processing Units (GPUs) using a mix of data parallelism (DP) [23] and tensor parallelism (TP) [26]. The workload can be represented as a Directed Acyclic Graph (DAG), where the vertices represent compute and communication operations, and edges indicate dependencies. As illustrated, the corresponding traffic matrix

simply records the total traffic volume exchanged, but not the order in which traffic becomes available. In the example workload graph, the DP communication operations become available only in the backward pass, while several TP exchanges and computations have to be executed beforehand. Therefore, scheduling these communication operations out of order would stall the entire workload. This highlights a critical flaw: optimizing for aggregate demand can actively harm the application’s progress.

Furthermore, this time-agnostic view also eliminates opportunities for compute-communication overlap. Modern distributed training heavily relies on this overlap to amortize network costs. As shown in the example schedules on the right hand side of Figure 1, a workload-aware scheduler such as FLUX is capable of actively aligning circuit reconfigurations and communication with ongoing compute phases. If we isolate the network schedule from the compute dependencies, we lose the ability to effectively pipeline these operations.

The same figure further illustrates how RotorNet [19] and a BvN-based schedule [15] lead to substantial inefficiencies. RotorNet is *demand-oblivious* because it rotates through a fixed set of circuits with a fixed reconfiguration period. The BvN-based schedule is *demand-aware* since it rotates through a fixed set of circuits (one per permutation matrix), but with varying reconfiguration periods based on the weight of each permutation matrix. These schedules trigger unnecessary circuit reconfigurations, since they cannot anticipate the workload’s state. FLUX, by contrast, reduces the total reconfiguration cost by reusing circuits and amortizing the reconfiguration delay behind compute.

This exposes a major difference in perspective. Rather than simply satisfying the communication demand, we must schedule that same communication demand to finish exactly when the compute needs it. This change in perspective is the main motivation for FLUX, and suggests a need to revisit how to schedule circuits for LLM training.

3 Circuit Scheduling as an Optimization Problem

In this section, we formalize workload-aware circuit scheduling as a MILP. The formulation takes two inputs: a system of n accelerators interconnected through an OCS network and a workload (a specific LLM training iteration).

System model. We consider a set $N = \{1, \dots, n\}$ of n accelerators (e.g., GPUs) interconnected through a set $S = \{1, \dots, s\}$ of s identical optical circuit switches. Each accelerator has exactly one physical link to each of the s switches, so it can maintain up to s simultaneous bidirectional circuits to other accelerators. Each switch is modeled as an optical $n \times n$

Table 1: Notation used throughout the problem formulation.

Symbol	Definition
$G = (V, E)$	Workload graph
$N = \{1, \dots, n\}$	Accelerators
$S = \{1, \dots, s\}$	Switches
(s_v, d_v)	src, dst pair of $v \in V$
C_v	Direct children of $v \in V$
$\mathcal{D}_v$	Descendants of $v \in V$
$\mathcal{A}_v$	Ancestors of $v \in V$
$\mathcal{P}_v$	Direct parents of $v \in V$
p_v	Transmission time of $v \in V$
$c_{u,v}$	Compute time between u and v
τ	Reconfiguration delay

crossbar: at any point in time, it can realize a set of simultaneous point-to-point circuits, with the only restriction being that no two circuits may share an endpoint at that switch. Setting up or changing a circuit takes a reconfiguration delay $\tau > 0$ s.

Workload model. As mentioned in Section 2, a distributed LLM training iteration has a highly temporal structure consisting of compute and communication operations and their dependencies. Let G be a DAG $G = (V, E)$ where each vertex $v \in V$ is an atomic point-to-point send between two accelerators and each edge $(u, v) \in E$ encodes a dependency: u must fully complete before v can begin. To encode the compute operations from the workload in the graph, each edge $(u, v) \in E$ additionally carries a non-negative weight $c_{u,v} \geq 0$, which represents the amount of compute time that must elapse between two send tasks $u, v \in V$. Each send task $v \in V$ has a fixed source-destination pair $(s_v, d_v) \in N \times N$, and a transmission time $p_v > 0$ that captures how long the corresponding send occupies the fabric (e.g., message size divided by link capacity).

3.1 Objective and Constraints

Given a workload graph and a system, we make two decisions per send task: at what time it starts, and on which switch it is executed. We introduce a continuous decision variable $t_v \geq 0$ for the start time of each task $v \in V$.

Objective function. The objective function minimizes the total completion time of the workload graph:

$$\min \max_{v \in V} (t_v + p_v). \quad (1)$$

Switch assignment. To capture on which switch a certain send task v will be executed, we introduce a binary decision variable $x_{v,s} \in \{0, 1\}$, which equals 1 if and only if v

is assigned to switch $s \in S$. Evidently, every task must be assigned to exactly one switch:

$$\sum_{s \in S} x_{v,s} = 1 \quad \forall v \in V. \quad (2)$$

DAG dependencies. The structure of the workload graph directly constrains the start times. If task u is a direct parent of task v , then v cannot start before u has finished transmitting and any intermediate compute has completed:

$$t_v \geq t_u + p_u + c_{u,v} \quad \forall (u, v) \in E. \quad (3)$$

These constraints fix the execution order of every pair of tasks with a direct edge in the graph, independently of which switch they are assigned to.

The remaining constraints capture the interactions between send tasks u, v that are sent on the same switch. More specifically, these constraints capture the physical limitations of circuit switching related to reconfiguration. When two tasks u, v share an endpoint (i.e., they share a source or destination) and are scheduled on the same switch, there should be a reconfiguration delay τ between them. Before formally defining this behavior in constraints, we first define a helper set of *potentially conflicting pairs* that collects all pairs of distinct tasks with at least one endpoint in common:

$$\mathcal{K} = \{(u, v) \mid u, v \in V, u \neq v, s_u = s_v \vee d_u = d_v \vee s_u = d_v \vee d_u = s_v\}.$$

For any such pair of tasks, a reconfiguration may be necessary if the tasks were to be scheduled on the same switch. We define the reconfiguration penalty incurred when task v executes after task u on the same switch:

$$R_{u,v} = \begin{cases} 0, & \text{if } (s_u, d_u) = (s_v, d_v), \\ \tau, & \text{otherwise.} \end{cases}$$

If the later task reuses the exact same circuit, there is no need for circuit reconfiguration. Otherwise, the switch must be reconfigured and the start time of the later task is delayed by τ .

Whether the relative execution order (i.e., u before v or v before u) of a conflicting pair (u, v) is already known depends on the DAG. We write $u \prec_G v$ (*precedence*) if and only if v is a descendant of u in G , in which case u always executes before v . Similarly, we write $u \parallel_G v$ (*incomparability*) if and only if neither task is an ancestor or descendant of the other, in which case their relative execution order becomes a scheduling decision. Based on this distinction, we partition the conflicting pairs into two disjoint sets:

$$\begin{aligned} \mathcal{K}^\prec &= \{(u, v) \in \mathcal{K} \mid u \prec_G v\}, \\ \mathcal{K}^\parallel &= \{(u, v) \in \mathcal{K} \mid u \parallel_G v\}. \end{aligned}$$

Fixed-order reconfiguration. For pairs in $\mathcal{K}^\prec$, the order is known, but the precedence constraints from Equation (3) do not yet account for the reconfiguration delay that arises when both tasks are placed on the same switch without reusing the same circuit. For every such pair and every switch, we add:

$$\begin{aligned} \forall (u, v) \in \mathcal{K}^\prec, \forall s \in S : \\ t_v \geq t_u + p_u + R_{u,v} - M(2 - x_{u,s} - x_{v,s}). \end{aligned} \quad (4)$$

In this set of constraints, we use the Big-M method [3], where we choose a sufficiently large constant M to deactivate conditional constraints, allowing us to write the constraints linearly. When u and v are placed on different switches, at least one of $x_{u,s}, x_{v,s}$ equals 0 and the subtracted Big-M term deactivates the constraint. When both tasks share switch s , the constraint enforces the reconfiguration overhead $R_{u,v}$. Practically speaking, we set the value of M to be the equivalent of scheduling all compute and communication tasks in sequence and reconfiguring between each task:

$$M = |V| \cdot \tau + \sum_{v \in V} p_v + \sum_{(u,v) \in E} c_{u,v}.$$

Free-order reconfiguration. For incomparable pairs in $\mathcal{K}^\parallel$, the scheduler must also decide their relative execution order. If u and v are placed on the same switch, then either u finishes (plus any required reconfiguration) before v starts, or vice versa. To express this linearly, we introduce, for each unordered pair $\{u, v\} \in \mathcal{K}^\parallel$, a binary sequencing variable $y_{u,v} \in \{0, 1\}$ that equals 1 if task u is scheduled before task v , and 0 otherwise. For every pair and every switch, we impose:

$$\begin{aligned} \forall \{u, v\} \in \mathcal{K}^\parallel, \forall s \in S : \\ t_v \geq t_u + p_u + R_{u,v} - M(3 - x_{u,s} - x_{v,s} - y_{u,v}) \\ t_u \geq t_v + p_v + R_{v,u} - M(2 - x_{u,s} - x_{v,s} + y_{u,v}). \end{aligned} \quad (5)$$

Whenever u and v are not both assigned to switch s , both inequalities are deactivated since tasks on different switches never interfere. When $x_{u,s} = x_{v,s} = 1$, then $y_{u,v}$ controls which of the two constraints is activated.

4 Evaluation

We empirically evaluate whether *workload-aware circuit scheduling* translates into end-to-end improvements under realistic training traces. In particular, we ask three questions: (1) how do different reconfiguration delays impact scheduling? (2) does workload awareness alleviate NIC buffering pressure? and (3) does minimizing the number of reconfigurations translate to performance improvements?

To answer these questions, we create FLUX, an implementation of the MILP from Section 3 in Gurobi [11] and use it to compute optimal schedules under our model. We then replay these schedules in a discrete-event network simulator

and compare FLUX against RotorNet [19] and a BvN-based demand-aware scheduler. Our results show that FLUX consistently achieves the lowest iteration time across the evaluated reconfiguration delays, while significantly reducing peak NIC buffer demand by more than three orders of magnitude relative to the other schedulers.

4.1 Simulation Setup

We evaluate FLUX using discrete-event network simulations built on an adapted version of ASTRA-sim [32]. We integrate an internal OCS simulator with a modified version of ASTRA-sim to enable circuit switching control. This OCS simulator takes as input, for each switch, a sequence of circuit states and the times at which the switch transitions between. For each workload and hardware configuration, we first solve the MILP from Section 3 with Gurobi to obtain a schedule, then convert it into this input format and simulate it.

Generating workload traces. We generate LLM training traces using an in-house workload execution trace generator. This tool generates Chakra [27] traces, a graph-based representation of AI/ML workloads, for a given model and system configuration, which we then feed to ASTRA-sim. For our evaluation, we use traces based on the Llama 3 8B model [9].

Hardware and network topology. We model a system setup with 8 GPUs and $s = 2$ optical circuit switches, following the same network topology as illustrated in Figure 1. Each GPU has one 800 Gbps bidirectional link to each switch, for two links per GPU in total. For the compute, we model an Nvidia A100 GPU using the hierarchical roofline model from Kelis [10]. Each GPU is equipped with a NIC with two links, one to each OCSs. We assume that each NIC has an unlimited amount of on-chip memory to store generated packets, allowing us to measure the effect of the circuit schedule on buffer requirements.

Circuit schedulers. We compare FLUX against RotorNet [19] and a BvN-based schedule. We implement RotorNet in two variants: a single-hop variant (RotorNet-Direct) and a variant using Valiant Load Balancing (VLB). Both variants use a 90% duty cycle, meaning that slot length increases with reconfiguration delay. For the BvN-based scheduler, we construct a traffic matrix from the workload and decompose it into permutation matrices with corresponding weights. We then use the permutation matrices as circuit configurations and their weights to determine slot lengths. Unlike RotorNet, the slot lengths in the BvN-based schedule are independent of the reconfiguration delay.

4.2 Performance Results

Figure 2 shows the end-to-end training iteration time, peak on-chip NIC memory usage, and number of reconfigurations

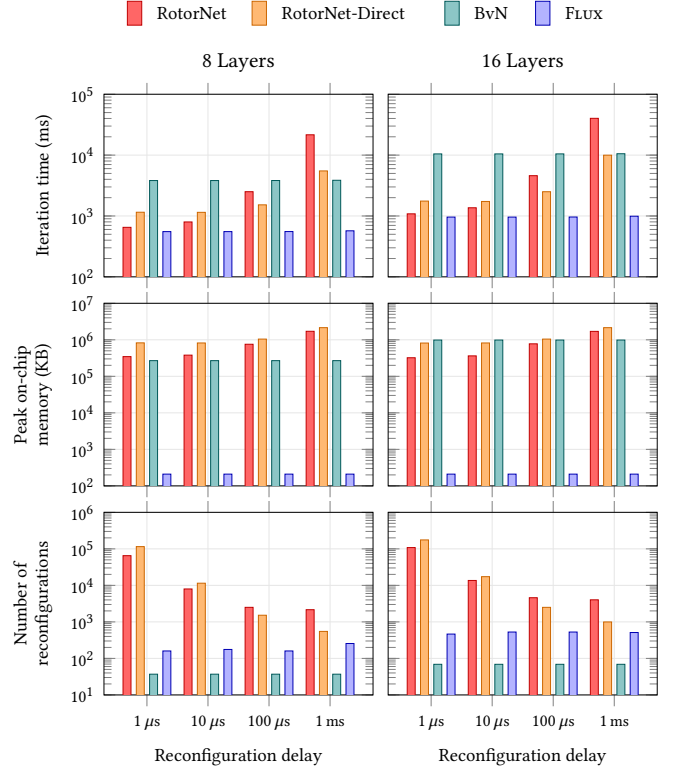


Figure 2: End-to-end training iteration time (top) and peak on-chip NIC memory usage (middle), and number of reconfigurations (bottom) across different reconfiguration delays for Llama 3 8B, 8/16 layers, TP4×DP2, 8 GPUs, 2 OCSs.

for 8 and 16 layers of Llama 3 8B trained on 8 GPUs, with reconfiguration delays ranging from 1 μ s to 1 ms.

The iteration time shows that RotorNet performs well at small reconfiguration delays, where its time slots are smaller and require less waiting for the right circuit. However, the iteration time grows with increasing reconfiguration delay. At higher delays, the demand-aware BvN schedule performs better due to its constant slot lengths. Nevertheless, FLUX consistently achieves the lowest iteration time, since it schedules circuits together with the workload, allowing communication and reconfiguration to be overlapped with compute.

A major improvement of FLUX is in peak on-chip NIC memory demand. RotorNet and BvN are unaware of the temporal structure of the workload, so communication tasks may become available while there is no suitable circuit to use. These tasks accumulate in the NIC packet buffers, resulting in high peak memory usage. In contrast, FLUX schedules circuits based on the entire workload structure, keeping buffer requirements low.

Finally, the number of reconfigurations performed in each experiment provides an interesting insight: a system that reconfigures less often does not necessarily perform better. At low reconfiguration delays, RotorNet performs substantially more reconfigurations than BvN and FLUX, yet achieves competitive iteration times. Similarly, BvN consistently performs fewer reconfigurations than FLUX, but has longer iteration times and substantially higher memory usage. Therefore, what matters is not how often the network reconfigures, but whether a reconfiguration enables useful circuits.

4.3 Discussion

Our experimental results demonstrate the effectiveness of *workload-aware circuit scheduling* and FLUX for LLM training. By considering the temporal structure of the workload, FLUX can overlap communication and reconfiguration with compute, while minimizing buffer requirements. We observe that FLUX can be effectively used across a wide variety of OCS switching technologies, from slower Micro Electro-Mechanical Systems (MEMS)-based switches with millisecond switching [28] to silicon photonics-based circuit switches with micro- or nanosecond reconfiguration delay [31].

FLUX currently relies on exact MILP solving, and solve time grows exponentially in the worst case with the number of binary variables. Since the number of communication tasks to schedule grows with world size (i.e., number of accelerators) and Transformer layer count, FLUX is currently limited to small-scale training jobs. Scaling to larger deployments will require the design of efficient heuristics or relaxations in place of exact MILP solving, which we leave as an important avenue for future work.

Another unresolved question is how FLUX can accommodate mixture-of-experts (MoE) models. MoE models generate non-deterministic and data-dependent communication patterns by routing tokens to different experts [5, 25]. FLUX assumes perfect knowledge of communication types and sizes: a realistic assumption for all types of parallelism used in LLM training and inference, with the exception of expert parallelism (EP). This is an important topic for future work.

5 Related Work

Circuit switching strategies have been widely studied, whether for general datacenter use [1, 2, 4, 15, 17–20] or for distributed training workloads [12, 13, 16, 21, 24, 30, 33]. However, many of these approaches do not exploit the deterministic and temporal nature of LLM training, the interaction between compute and communication, and the cost of reconfiguration.

Workload-aware circuit scheduling. Existing circuit scheduling approaches typically rely on deriving a schedule

from an aggregate traffic demand matrix, obtained by summing or averaging communication volume over some time window (e.g., a training iteration) [4, 13, 15, 20, 24, 30, 33]. For LLM training, where compute and communication operations have a specific execution order, aggregating demand over time discards this ordering. Additionally, most of this work treats scheduling as a purely networking problem, decoupled from compute, missing opportunities to overlap communication with compute. In contrast, FLUX schedules circuit reconfigurations and communication while explicitly taking into account when compute occurs.

Complexity of reconfiguration. Circuit reconfiguration induces a non-negligible reconfiguration delay, and its cost varies substantially across switching technologies, from nanoseconds [2] to milliseconds [28]. A large fraction of existing work does not consider this delay as an explicit parameter of the scheduling problem, instead assuming it to be negligible or fixing it to a value tied to a particular hardware platform. Treating reconfiguration delay as a fixed constant rather than a first-class parameter of the problem obscures the trade-off between the benefit of adapting the topology and the overhead incurred by doing so. Since FLUX takes the reconfiguration delay as a parameter and is aware of the workload structure, it naturally creates opportunities for circuit reuse and amortizing the reconfiguration cost.

6 Conclusion and Future Work

In this paper, we make the case for *workload-aware circuit scheduling* for LLM training. We argue that existing demand-aware traffic matrix-based reconfiguration schemes are not suitable for these highly coupled and structured workloads, and that it is important to consider the reconfiguration delay of the system, the temporal dependencies of the workload, and the compute when scheduling the communication. To show this, we presented FLUX, a framework that formalizes *workload-aware circuit scheduling*, and showed that it naturally leads to circuit reuse, amortization of reconfiguration and communication costs, and significantly lower NIC buffer requirements.

In future work, our objective will be to turn FLUX from a powerful formalization into a tool that can be used to schedule large and realistic problem sizes. This encompasses the design of intelligent heuristics, support for MoE models, and an extensive evaluation campaign.

References

- [1] Vamsi Addanki, Chen Avin, and Stefan Schmid. 2023. Mars: Near-Optimal Throughput with Shallow Buffers in Reconfigurable Datacenter Networks. *Proc. ACM Meas. Anal. Comput. Syst.* 7, 1 (March 2023), 2:1–2:43. doi: 10.1145/3579312.
- [2] Hitesh Ballani, Paolo Costa, Raphael Behrendt, Daniel Cletheroe, Istvan Haller, Krzysztof Jozwiak, Fotini Karinou, Sophie Lange, Kai

- Shi, Benn Thomsen, and Hugh Williams. 2020. Sirius: A Flat Datacenter Network with Nanosecond Optical Switching. In *Proceedings of the ACM SIGCOMM 2020 Conference (SIGCOMM '20)*. Association for Computing Machinery, New York, NY, USA, 782–797. doi: 10.1145/3387514.3406221.
- [3] Mokhtar S. Bazaraa, John J. Jarvis, and J. J. Jarvis. 1977. *Linear Programming and Network Flows*. Wiley, New York.
- [4] Nathan Farrington, George Porter, Sivasankar Radhakrishnan, Hamid Hajabdolali Bazzaz, Vikram Subramanya, Yeshaiah Fainman, George Papen, and Amin Vahdat. 2010. Helios: A Hybrid Electrical/Optical Switch Architecture for Modular Data Centers. In *Proceedings of the ACM SIGCOMM 2010 Conference (SIGCOMM '10)*. Association for Computing Machinery, New York, NY, USA, 339–350. doi: 10.1145/1851182.1851223.
- [5] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *The Journal of Machine Learning Research* 23, 1 (Jan. 2022), 120:5232–120:5270. <https://dl.acm.org/doi/10.5555/3586589.3586709>
- [6] Adithya Gangidi, Rui Miao, Shengbao Zheng, Sai Jayesh Bondu, Guilherme Goes, Hany Morsy, Rohit Puri, Mohammad Riftadi, Ashmitha Jeevaraj Shetty, Jingyi Yang, Shuqiang Zhang, Mikel Jimenez Fernandez, Shashidhar Gandham, and Hongyi Zeng. 2024. RDMA over Ethernet for Distributed Training at Meta Scale. In *Proceedings of the ACM SIGCOMM 2024 Conference*. ACM, Sydney NSW Australia, 57–70. doi: 10.1145/3651890.3672233.
- [7] Garrett Birkhoff. 1946. Tres Observaciones Sobre El Algebra Lineal. *Univ. Nac. Tucuman, Ser. A* 5 (1946), 147–154.
- [8] Amir Gholami, Zhewei Yao, Sehoon Kim, Coleman Hooper, Michael W. Mahoney, and Kurt Keutzer. 2024. AI and Memory Wall. *IEEE Micro* 44, 3 (May 2024), 33–39. doi: 10.1109/MM.2024.3373763.
- [9] Aaron Grattafiori et al. 2024. The Llama 3 Herd of Models. arXiv:2407.21783 [cs]
- [10] Wenzhe Guo, Joyjit Kundu, Uras Tos, Weijiang Kong, Giuliano Sisto, Timon Evenblij, and Manu Perumkunnil. 2025. Keeping up with Large Language Models: A Holistic Methodology of Compute, Memory, Communication, and Cost Modeling. In *2025 IEEE International Symposium on Workload Characterization (IISWC)*. 116–126. doi: 10.1109/IISWC66894.2025.00019.
- [11] Gurobi Optimization, LLC. 2026. Gurobi Optimizer. <https://www.gurobi.com>. Version 13.0.2.
- [12] Norm Jouppi, George Kurian, Sheng Li, Peter Ma, Rahul Nagarajan, Lifeng Nai, Nishant Patil, Suvinay Subramanian, Andy Swing, Brian Towles, Clifford Young, Xiang Zhou, Zongwei Zhou, and David A Patterson. 2023. TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings. In *Proceedings of the 50th Annual International Symposium on Computer Architecture*. ACM, Orlando FL USA, 1–14. doi: 10.1145/3579371.3589350.
- [13] Mehrdad Khani, Manya Ghobadi, Mohammad Alizadeh, Ziyi Zhu, Madeleine Glick, Keren Bergman, Amin Vahdat, Benjamin Klenk, and Eiman Ebrahimi. 2021. SiP-ML: High-Bandwidth Optical Network Interconnects for Machine Learning Training. In *Proceedings of the 2021 ACM SIGCOMM 2021 Conference (SIGCOMM '21)*. Association for Computing Machinery, New York, NY, USA, 657–675. doi: 10.1145/3452296.3472900.
- [14] Wenxue Li, Xiangzhou Liu, Yuxuan Li, Yilun Jin, Han Tian, Zhizhen Zhong, Guyue Liu, Ying Zhang, and Kai Chen. 2024. Understanding Communication Characteristics of Distributed Training. In *Proceedings of the 8th Asia-Pacific Workshop on Networking (APNet '24)*. Association for Computing Machinery, New York, NY, USA, 1–8. doi: 10.1145/3663408.3663409.
- [15] He Liu, Matthew K. Mukerjee, Conglong Li, Nicolas Feltman, George Papen, Stefan Savage, Srinivasan Seshan, Geoffrey M. Voelker, David G. Andersen, Michael Kaminsky, George Porter, and Alex C. Snoeren. 2015. Scheduling Techniques for Hybrid Circuit/Packet Networks. In *Proceedings of the 11th ACM Conference on Emerging Networking Experiments and Technologies (CoNEXT '15)*. Association for Computing Machinery, New York, NY, USA, 1–13. doi: 10.1145/2716281.2836126.
- [16] Hong Liu, Ryohei Urata, Kevin Yasumura, Xiang Zhou, Roy Bannan, Jill Berger, Pedram Dashti, Norm Jouppi, Cedric Lam, Sheng Li, Erji Mao, Daniel Nelson, George Papen, Mukarram Tariq, and Amin Vahdat. 2023. Lightwave Fabrics: At-Scale Optical Circuit Switching for Datacenter and Machine Learning Systems. In *Proceedings of the ACM SIGCOMM 2023 Conference*. ACM, New York NY USA, 499–515. doi: 10.1145/3603269.3604836.
- [17] William M. Mellette, Rajdeep Das, Yibo Guo, Rob McGuinness, Alex C. Snoeren, and George Porter. 2020. Expanding across Time to Deliver Bandwidth Efficiency and Low Latency. In *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. 1–18. <https://www.usenix.org/conference/nsdi20/presentation/mellette>
- [18] William M. Mellette, Alex Forencich, Rukshani Athapathu, Alex C. Snoeren, George Papen, and George Porter. 2024. Realizing RotorNet: Toward Practical Microsecond Scale Optical Networking. In *Proceedings of the ACM SIGCOMM 2024 Conference*. ACM, Sydney NSW Australia, 392–414. doi: 10.1145/3651890.3672273.
- [19] William M. Mellette, Rob McGuinness, Arjun Roy, Alex Forencich, George Papen, Alex C. Snoeren, and George Porter. 2017. RotorNet: A Scalable, Low-complexity, Optical Datacenter Network. In *Proceedings of the Conference of the ACM Special Interest Group on Data Communication*. ACM, Los Angeles CA USA, 267–280. doi: 10.1145/3098822.3098838.
- [20] George Porter, Richard Strong, Nathan Farrington, Alex Forencich, Pang Chen-Sun, Tajana Rosing, Yeshaiah Fainman, George Papen, and Amin Vahdat. 2013. Integrating Microsecond Circuit Switching into the Data Center. In *Proceedings of the ACM SIGCOMM 2013 Conference on SIGCOMM (SIGCOMM '13)*. Association for Computing Machinery, New York, NY, USA, 447–458. doi: 10.1145/2486001.2486007.
- [21] Leon Poutievski, Omid Mashayekhi, Joon Ong, Arjun Singh, Mukarram Tariq, Rui Wang, Jianan Zhang, Virginia Beaugregard, Patrick Conner, Steve Gribble, Rishi Kapoor, Stephen Kratzner, Nanfang Li, Hong Liu, Karthik Nagaraj, Jason Ornstein, Samir Sawhney, Ryohei Urata, Lorenzo Vicisano, Kevin Yasumura, Shidong Zhang, Junlan Zhou, and Amin Vahdat. 2022. Jupiter Evolving: Transforming Google’s Datacenter Network via Optical Circuit Switches and Software-Defined Networking. In *Proceedings of the ACM SIGCOMM 2022 Conference (SIGCOMM '22)*. Association for Computing Machinery, New York, NY, USA, 66–85. doi: 10.1145/3544216.3544265.
- [22] Kun Qian, Yongqing Xi, Jiamin Cao, Jiaqi Gao, Yichi Xu, Yu Guan, Binzhang Fu, Xuemei Shi, Fangbo Zhu, Rui Miao, Chao Wang, Peng Wang, Pengcheng Zhang, Xianlong Zeng, Eddie Ruan, Zhiping Yao, Ennan Zhai, and Dennis Cai. 2024. Alibaba HPN: A Data Center Network for Large Language Model Training. In *Proceedings of the ACM SIGCOMM 2024 Conference*. ACM, Sydney NSW Australia, 691–706. doi: 10.1145/3651890.3672265.
- [23] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. ZeRO: Memory Optimizations toward Training Trillion Parameter Models. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC '20)*. IEEE Press, Atlanta, Georgia, 1–16. <https://dl.acm.org/doi/10.5555/3433701.3433727>
- [24] Sundararajan Renganathan and Nick McKeown. 2025. Chronos: Prescheduled Circuit Switching for LLM Training. In *Proceedings of*

- the 2nd Workshop on Networks for AI Computing (NAIC '25)*. Association for Computing Machinery, New York, NY, USA, 89–97. doi: 10.1145/3748273.3749210.
- [25] Noam Shazeer, *Azalia Mirhoseini, *Krzysztof Maziarczyk, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=B1ckMDqlg>
- [26] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2020. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. arXiv:1909.08053 [cs]
- [27] Srinivas Sridharan, Taekyung Heo, Louis Feng, Zhaodong Wang, Matt Bergeron, Wenyin Fu, Shengbao Zheng, Brian Coutinho, Saeed Rashidi, Changhai Man, and Tushar Krishna. 2023. Chakra: Advancing Performance Benchmarking and Co-design Using Standardized Execution Traces. arXiv:2305.14516 [cs.LG]
- [28] Ryohei Urata, Hong Liu, Kevin Yasumura, Erji Mao, Jill Berger, Xiang Zhou, Cedric Lam, Roy Bannon, Darren Hutchinson, Daniel Nelson, Leon Poutievski, Arjun Singh, Joon Ong, and Amin Vahdat. 2022. Mission Apollo: Landing Optical Circuit Switching at Datacenter Scale. arXiv:2208.10041 [cs.NI]
- [29] John Von Neumann. 1953. A Certain Zero-Sum Two-Person Game Equivalent to the Optimal Assignment Problem. *Contributions to the Theory of Games II; Annals of Mathematics Studies* 28 (1953), 5–12.
- [30] Weiyang Wang, Moein Khazraee, Zhizhen Zhong, Manya Ghobadi, Zhihao Jia, Dheevatsa Mudigere, Ying Zhang, and Anthony Kewitsch. 2023. TopoOpt: Co-optimizing Network Topology and Parallelization Strategy for Distributed Training Jobs. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. 739–767. <https://www.usenix.org/conference/nsdi23/presentation/wang-weiyang>
- [31] Yuyang Wang, Songli Wang, Robert Parsons, Swarnava Sanyal, Vignesh Gopal, Asher Novick, Anthony Rizzo, Michal Lipson, Alexander L. Gaeta, and Keren Bergman. [n. d.]. Co-Designed Silicon Photonics Chip I/O for Energy-Efficient Petascale Connectivity. 15, 8 ([n. d.]), 1581–1591. doi: 10.1109/TCPMT.2024.3492189.
- [32] William Won, Taekyung Heo, Saeed Rashidi, Srinivas Sridharan, Sudarshan Srinivasan, and Tushar Krishna. 2023. ASTRA-sim2.0: Modeling Hierarchical Networks and Disaggregated Systems for Large-model Training at Scale. In *2023 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. 283–294. doi: 10.1109/ISPASS57527.2023.00035.
- [33] Zhenguo Wu, Benjamin Klenk, Larry Dennison, and Keren Bergman. 2025. ACTINA: Adapting Circuit-Switching Techniques for AI Networking Architectures. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. ACM, St. Louis MO USA, 1211–1222. doi: 10.1145/3712285.3759842.